

可信天气数据改造说明

适用版本：v1.0.0 文档日期：2026-09-19 阅读对象：刚开始学习编程、后端、API 或 AI Agent 的开发者
更新说明：补充旅行规划失败语义、分层异常处理和相关测试

1. 这次修改解决了什么问题？

这个项目可以根据用户填写的城市、日期和旅行偏好生成旅行计划。旅行计划里还会显示每天的天气。

修改前，页面虽然能显示天气，但存在一个严重问题：**页面上的天气不一定全部来自高德地图。**

例如：

- 用户创建了 7 天旅行计划；
- 高德地图实际只返回了 4 天预报；
- 行程规划 Agent 的提示词却要求“每一天都必须有天气”；
- 大语言模型为了完成任务，可能自行补出后 3 天的天气；
- 页面会把真实天气和模型补写的天气以相同样式展示。

用户看到 7 张天气卡片时，很容易误以为这 7 天都来自高德地图。实际上，后面的数据可能只是大语言模型根据常识生成的内容，并不是真实预报。

这次改造的核心目标是：

高德提供多少可信天气，系统就展示多少；没有可靠数据的日期明确提示用户，而不是让 AI 猜测。

2. 先认识几个基础概念

2.1 FastAPI 是什么？

FastAPI 是本项目的后端 Web 框架。浏览器或 Postman 请求：

```
GET /api/map/weather?city=北京
```

FastAPI 会找到对应的 Python 路由函数，把查询参数 `city=北京` 交给它处理，最后把 Python 数据转换成 HTTP JSON 响应。

可以把 FastAPI 理解为餐厅的“前台服务员”：它接收用户需求，再把任务交给后面的业务模块。

2.2 Service 是什么？

Service 是专门处理某类业务的服务层。本项目中的 `AmapService` 负责和高德地图 MCP 沟通。

它的职责包括：

- 调用高德工具；
- 接收原始结果；
- 解析数据；

- 转换成项目内部统一的数据模型；
- 把异常继续交给上层处理。

可以把 Service 理解为餐厅的“厨房”：前台不需要知道菜是怎么做的，只需要拿到最终结果。

2.3 MCP 是什么？

MCP（Model Context Protocol）是一种让应用或 AI 调用外部工具的协议。

本项目通过：

```
uvx amap-mcp-server
```

启动高德地图 MCP 服务，并调用其中的：

```
maps_weather
```

工具。

2.4 JSON 是什么？

JSON 是一种常见的数据交换格式，例如：

```
{
  "city": "北京市",
  "forecasts": [
    {
      "date": "2026-09-17",
      "dayweather": "阴",
      "daytemp": "28"
    }
  ]
}
```

它看起来像 Python 字典，但从外部工具返回时，经常只是普通字符串。程序必须先解析，才能按照字段读取内容。

2.5 Pydantic Model 是什么？

Pydantic Model 可以理解为“数据格式检查员”。本项目使用 `WeatherInfo` 规定一条天气必须有哪些字段：

<code>date</code>	日期
<code>day_weather</code>	白天天气
<code>night_weather</code>	夜间天气
<code>day_temp</code>	白天气温
<code>night_temp</code>	夜间气温

```
wind_direction    风向
wind_power        风力
```

Pydantic 不只是保存数据，还会检查和转换数据。例如高德返回的温度是字符串：

```
"28"
```

经过 [WeatherInfo](#) 的温度校验器后，会变成整数：

```
28
```

2.6 LLM 和 Agent 是什么？

LLM 是大语言模型。Agent 是在 LLM 基础上增加提示词、工具和任务流程的程序。

LLM 很擅长生成自然语言和规划行程，但它不是天气数据库。当真实数据不足时，LLM 仍可能生成“看起来合理”的天气，这种现象通常叫作“幻觉”。

3. 修改前的数据链路

修改前，独立天气接口虽然调用了高德 MCP，但没有真正解析结果：

```
FastAPI 路由
  ↓
AmapService.get_weather()
  ↓
调用 maps_weather
  ↓
收到原始字符串
  ↓
固定 return []
  ↓
API 返回 data: []
```

与此同时，旅行规划使用 Weather Agent 查询天气，再把原始文本交给 Planner Agent：

```
Weather Agent
  ↓
高德 MCP 原始文本
  ↓
Planner Agent
  ↓
LLM 生成整份 TripPlan
```

↓
前端展示 LLM 输出的 weather_info

这里存在两个问题：

1. `/api/map/weather` 永远返回空列表，独立天气接口没有真正完成。
2. Planner 可以修改、遗漏或补写天气，不能保证页面数据忠于高德。

4. 修改后的完整数据链路

修改后，数据会经过下面的步骤：

```
GET /api/map/weather?city=北京
↓
FastAPI 路由接收 city
↓
AmapService.get_weather(city)
↓
MCPTool.run(... maps_weather ...)
↓
高德 MCP 返回带说明文字的字符串
↓
提取字符串中的 JSON
↓
json.loads() 转成 Python dict
↓
读取 forecasts 数组
↓
逐条创建 WeatherInfo
↓
返回 List[WeatherInfo]
↓
WeatherResponse 包装
↓
FastAPI 输出 JSON
```

旅行规划链路则变为：

```
AmapService 返回 List[WeatherInfo]
├─ 序列化为 JSON 文本，供 Planner 参考
└─ 保留原始模型列表，作为可信数据
↓
Planner Agent 生成 TripPlan
↓
按照旅行日期筛选可信天气
↓
覆盖 LLM 生成的 weather_info
```

↓
前端只展示高德真实提供的天气

5. 后端天气 Service 做了哪些修改？

相关文件：

```
backend/app/services/amap_service.py
```

5.1 调用高德 MCP

Service 调用：

```
action: call_tool
tool_name: maps_weather
arguments: { city: 用户输入的城市 }
```

实际测试发现，`MCPTool.run()` 返回的是 Python `str`，不是 `dict`。

返回内容大致是：

```
工具 'maps_weather' 执行结果：
{
  "city": "北京市",
  "forecasts": [...]
}
```

前面的“工具执行结果”是 HelloAgents 框架添加的说明文字，因此不能直接对整个字符串调用 `json.loads()`。

5.2 提取 JSON 主体

程序查找第一个 `{` 和最后一个 `}`，截取中间的完整 JSON 对象。

如果返回值里没有 JSON，程序会抛出异常：

```
响应中未找到JSON数据
```

这样可以避免把 MCP 连接失败、城市无效等错误误认为正常天气。

5.3 把 JSON 字符串解析成字典

程序使用 `json.loads()` 完成转换：

JSON字符串 → Python dict

解析后的顶层结构包括：

```
city
forecasts
```

其中 `forecasts` 是一个列表，每一项代表一天的天气。

5.4 检查返回结构

程序检查：

- `forecasts` 是否是列表；
- 列表是否为空；
- 列表中的每一项是否是字典。

这些检查的意义是尽早发现外部数据格式异常。如果不检查，错误可能一直传到前端，排查起来会更加困难。

5.5 映射字段

高德字段名和项目字段名并不完全相同：

高德字段	项目字段	含义
<code>date</code>	<code>date</code>	日期
<code>dayweather</code>	<code>day_weather</code>	白天天气
<code>nightweather</code>	<code>night_weather</code>	夜间天气
<code>daytemp</code>	<code>day_temp</code>	白天气温
<code>nighttemp</code>	<code>night_temp</code>	夜间气温
<code>daywind</code>	<code>wind_direction</code>	白天风向
<code>daypower</code>	<code>wind_power</code>	白天风力

本次实现选择白天风向和白天风力填入统一字段，因为当前 `WeatherInfo` 只有一组风向、风力字段，没有分别保存白天和夜间数据。

5.6 创建 `WeatherInfo`

每一项高德天气字典都会创建一个新的 `WeatherInfo` 对象。

这不是把原字典“原地变形”，而是：

```
高德天气 dict
↓ 读取和映射字段
新的 WeatherInfo 对象
```

原始字典里的温度仍是字符串，而新模型中的温度已经由 Pydantic 转成整数。

5.7 返回模型列表

所有天气转换完成后，Service 返回：

```
List[WeatherInfo]
```

FastAPI 会自动把 Pydantic 对象序列化为前端能读取的 JSON。

5.8 正确传播错误

修改前，异常被捕获后返回空列表：

```
查询失败 → return [] → API仍显示success: true
```

这会让调用方无法区分“真的没有数据”和“服务已经失败”。

修改后，Service 记录错误并重新抛出：

```
查询失败 → raise → 上层决定如何处理
```

对于独立天气接口，路由会返回 HTTP 500 和错误说明，不再伪装成查询成功。

6. Pydantic 模型发挥了什么作用？

相关文件：

```
backend/app/models/schemas.py
```

本次没有重新设计 `WeatherInfo`，而是复用了项目已有模型。

其温度校验器会处理：

```
"28"    → 28  
"28°C"  → 28  
"28°C"  → 28
```

如果温度字符串无法转换，当前实现会使用 0。

这样做的好处是，温度清洗逻辑集中在模型中，Service 不需要到处重复写字符串处理代码。

7. 旅行规划链路做了哪些修改？

相关文件：

```
backend/app/agents/trip_planner_agent.py
```

7.1 不再通过 Weather Agent 获取天气

旧流程中的 Weather Agent 被移除。旅行规划器现在直接调用：

```
AmapService.get_weather(city)
```

这样能直接获得已经校验过的 `List[WeatherInfo]`，减少一次不必要的 LLM 解释过程。

7.2 为什么还要把模型转换成 JSON 字符串？

Planner Agent 接收的是文本提示词，不能直接读取 Python 模型对象。

因此程序会创建一份供 Planner 阅读的 JSON 文本：

```
WeatherInfo  
  ↓ model_dump()  
dict  
  ↓ json.dumps()  
JSON字符串
```

这里没有把数据转换回高德格式。转换后的 JSON 使用的仍然是项目字段，例如 `day_weather` 和 `day_temp`。

同时，原来的 `weather_info_list` 会继续保留，后面用于最终可信覆盖。

7.3 修改 Planner 提示词

旧提示词要求：

`weather_info`数组必须包含每一天的天气信息

当行程有 7 天、高德只有 4 天时，这条要求会迫使 LLM 补写 3 天天气。

新提示词明确要求：

- 只能使用输入中实际存在的天气；
- 不得推测或补全不存在日期的数据；
- 不得修改天气、温度、风向和风力；
- 没有天气的旅行日期直接省略。

7.4 为什么修改提示词还不够？

提示词只是对 LLM 的语言要求，不是程序的强制规则。模型大多数时候会遵守，但不能保证每一次都遵守。

因此项目增加了第二层保障：代码覆盖。

7.5 按旅行日期筛选天气

高德返回的是当前可预报日期，但用户旅行日期可能只覆盖其中一部分，也可能完全不重叠。

程序只保留满足下面条件的天气：

旅行开始日期 $\leq$ 天气日期 $\leq$ 旅行结束日期

因为项目日期统一使用 `YYYY-MM-DD` 格式，所以当前代码可以进行字符串比较。

7.6 用可信数据覆盖 LLM 输出

Planner 生成 `TripPlan` 后，程序会把其中的 `weather_info` 替换为筛选后的高德天气：

```
LLM weather_info
↓ 被替换
高德可信 weather_info
```

这样即使 LLM：

- 补出了更多日期；
- 修改了气温；
- 改写了天气；
- 生成了完全不存在的日期；

最终返回前端的天气仍然只能来自 Service。

这就是“提示词约束 + 代码强制”的双重保障。

7.7 天气失败时继续生成行程

独立天气接口的核心任务就是查天气，所以天气失败时返回错误是合理的。

旅行规划则不同。即使天气服务暂时不可用，用户仍然可以获得景点、酒店和餐饮计划。

因此旅行规划器会局部捕获天气异常：

```
天气成功 → 使用真实天气生成行程
天气失败 → weather_info为空，但继续生成行程
```

最后的可信覆盖会保证：即使天气查询失败后 LLM 自行生成了天气，也会被空列表覆盖。

8. 前端做了哪些修改？

相关文件：

```
frontend/src/views/Result.vue
```

前端会比较：

```
真实天气数量 < 行程天数
```

如果条件成立，就显示黄色提示，例如：

```
当前仅提供了 4 天可靠天气预报。
本次行程共 6 天。未显示的日期超出当前天气预报范围，请临近出发时再次查询。
```

提示放在天气卡片外部，因此即使天气数量为 0，用户仍能看到原因说明。

如果不加提示，用户看到天气卡片少于旅行天数时，可能误以为：

- 页面加载失败；
- 后端漏掉了数据；
- 程序出现了 Bug。

明确提示能让系统限制变得透明。

9. 为什么高德只有 4 天，旅行却可以超过 4 天？

旅行天数和天气预报范围是两件不同的事。

旅行计划可以生成 7 天、10 天甚至更长；但天气服务只能返回它当前支持的预报范围。程序不能通过增加 `travel_days` 参数凭空获得更远日期的真实天气。

正确产品逻辑是：

旅行计划：可以覆盖全部旅行日期
天气预报：只展示外部服务真实提供的日期
其他日期：提示暂时没有可靠预报

临近出发日期时重新查询，才可能获得后续日期的新预报。

10. 如果不做这次修改，行不行？

程序表面上仍然可以运行，但会存在以下问题。

10.1 独立天气接口没有实际价值

虽然调用了 MCP，最终却固定返回空列表。其他功能无法复用结构化天气。

10.2 用户可能把虚构数据当成真实预报

页面没有区分“高德数据”和“LLM 生成数据”，这会损害用户信任。

10.3 错误会被伪装成成功

外部服务失败时返回 `success: true, data: []`，会让排查问题变得困难。

10.4 LLM 会承担不适合它的工作

LLM 适合根据天气安排室内或室外活动，不适合创造天气事实。

10.5 后续扩展困难

如果未来需要缓存天气、定时刷新、替换天气提供商或编写测试，没有结构化 Service 层会让改动更加混乱。

因此，“不修改”只适合临时演示，不适合作为真实旅行助手继续发展。

11. 修改后有哪些好处？

11.1 数据更可信

前端展示的逐日天气有明确来源，不再由 LLM 猜测。

11.2 职责更清晰

高德 Service：提供天气事实
Pydantic：检查和统一数据
Planner Agent：根据事实规划行程
FastAPI：对外提供接口
Vue 前端：展示结果和限制说明

每层只负责自己擅长的事情。

11.3 错误更容易排查

Service、路由和 Planner 对错误有明确处理方式，不再用空列表掩盖所有问题。

11.4 系统更诚实

数据不足时告诉用户“暂无可靠预报”，比生成完整但虚假的内容更有价值。

11.5 更容易测试和复用

其他功能可以直接调用 `AmapService.get_weather()`，获得统一的 `List[WeatherInfo]`。

11.6 降低不必要的 LLM 依赖

天气数据不再需要 Weather Agent 解释，减少了一次模型参与和潜在的不确定性。

12. 如何验证修改成功？

12.1 正常天气接口

请求：

```
GET /api/map/weather?city=北京
```

预期：

- HTTP 200；
- `success` 为 `true`；
- `data` 包含高德返回的天气；
- 温度在 JSON 中是数字。

12.2 无效城市

请求：

```
GET /api/map/weather?city=123
```

预期：

- 无法获得有效 JSON 时返回 HTTP 500；
- 错误信息说明天气查询失败；
- 不再返回“成功 + 空列表”。

12.3 长行程

生成 6 天行程，而高德只返回 4 天天气。

预期：

- 行程仍有 6 天；

- 页面只显示 4 张真实天气卡片；
- 页面显示天气覆盖不足提示；
- 不生成剩余 2 天的虚构天气。

12.4 天气服务失败时生成行程

预期：

- 天气查询失败日志出现；
- 程序继续搜索酒店并生成计划；
- 最终 `weather_info` 为空；
- LLM 即使生成天气，也会被代码覆盖清空。

13. 当前实现仍有哪些限制？

这次修改提高了可靠性，但并不代表天气功能已经完美。

13.1 预报范围有限

当前高德 MCP 实际返回约 4 天预报，长行程无法一次获得每天的真实天气。

13.2 只保存一组风向和风力

高德返回白天和夜间两组风向、风力，当前模型只保存白天数据。

13.3 日期仍是字符串

项目依赖统一的 `YYYY-MM-DD` 格式进行日期筛选。未来可以改成真正的日期类型并增加开始、结束日期校验。

13.4 MCP 返回解析依赖花括号位置

当前通过第一个 `{` 和最后一个 `}` 提取 JSON，适用于已经观察到的返回格式。未来框架返回结构变化时，可能需要更稳定的结构化接口。

13.5 温度解析失败会变成 0

这可能掩盖外部数据异常。未来可以考虑让无效温度变为 `None` 或直接触发校验错误。

13.6 尚未持久化和自动刷新

旅行计划保存在浏览器 `sessionStorage`。系统还不能在预报范围更新后自动重新查询天气。

14. 后续可以怎样继续改进？

建议按优先级逐步进行：

1. 为 `AmapService.get_weather()` 编写单元测试；
2. 使用 `date` 类型校验日期和旅行范围；
3. 将外部服务异常区分为无效城市、连接失败和数据格式错误；
4. 在天气数据中记录来源和查询时间；
5. 支持用户手动刷新天气；

6. 对较长行程在临近日期时重新查询；
7. 如果业务确实需要更长预报，再评估其他天气数据源；
8. 将“季节气候建议”和“逐日真实天气”分开显示。

15. 这次修改最重要的经验

经验一：能生成不等于真实

LLM 可以快速生成完整内容，但天气、价格、库存等事实数据必须依赖可信数据源。

经验二：提示词不能代替程序校验

提示词可以减少错误，但真正可靠的系统还需要代码检查和强制覆盖。

经验三：外部数据必须经过适配层

外部工具字段不应该直接散落到整个项目。Service 和 Pydantic 模型负责把外部格式转换成项目统一格式。

经验四：不同入口可以采用不同失败策略

天气接口失败时应该报错；旅行规划中的天气失败时，可以继续生成无天气行程。错误处理应根据业务场景决定。

经验五：诚实展示限制也是功能的一部分

“暂无可靠天气预报”不是系统无能，而是对数据边界的准确表达。对真实产品来说，可信往往比看起来完整更重要。

16. 第一阶段补充改造：失败不能再伪装成成功

天气数据闭环完成后，项目继续处理了一个更通用的可靠性问题：当 MCP、LLM、JSON 解析或计划校验失败时，旧代码会返回一份备用行程。

旧流程大致如下：

```
任意步骤出现异常
↓
捕获 Exception
↓
调用 _create_fallback_plan()
↓
生成“某城市景点1”等占位景点
↓
使用北京附近的固定坐标
↓
API 返回 success: true
```

这种处理方式虽然能让页面继续显示内容，但会产生比直接报错更严重的问题：用户无法区分真实规划和程序临时编造的占位数据。

因此本阶段删除了虚构备用方案，改成：

```
成功并通过校验 → 返回 TripPlan
外部服务失败    → 抛出外部服务异常
Agent 输出错误  → 抛出 Agent 输出异常
业务规则失败    → 抛出计划校验异常
未知程序错误    → 返回内部错误
```

核心原则是：

没有可信结果时明确失败，不用虚构结果维持“看起来成功”。

17. 为什么要建立异常类型？

Python 中最宽泛的异常捕获方式是：

```
except Exception:
    ...
```

它可以捕获几乎所有普通运行异常，但无法说明问题究竟发生在哪里。例如下面几种情况都会进入同一个分支：

- 高德 MCP 无法连接；
- LLM 服务超时；
- Agent 返回普通文本而不是 JSON；
- JSON 字段缺失；
- Agent 返回重复 POI；
- 代码本身出现了未预料的 Bug。

如果这些情况全部返回 HTTP 500，前端和开发者就无法采取有针对性的处理。

项目新增了：

```
backend/app/exceptions.py
```

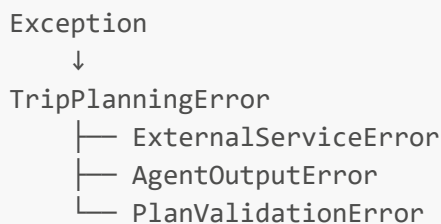
其中定义了四类业务异常。

17.1 TripPlanningError

```
class TripPlanningError(Exception):
    """旅行规划基础异常。"""
```

这是旅行规划异常的共同父类。它的作用是把“已经预料到的旅行规划失败”与普通程序 Bug 区分开。

继承关系如下：



API 层既可以分别捕获三个具体异常，也可以用 `TripPlanningError` 处理尚未细分的业务失败。

17.2 ExternalServiceError

该异常表示项目依赖的外部能力没有正常工作，例如：

- 高德 POI 搜索失败；
- 景点筛选模型无法调用；
- 行程规划模型无法调用；
- MCP 服务没有启动；
- 网络连接失败或第三方服务超时。

它表达的是：

请求本身可能没有问题，失败发生在外部依赖。

例如景点搜索失败时，编排层会把底层异常转换为：

```
raise ExternalServiceError(
    "景点数据服务暂时不可用"
) from error
```

`from error` 会保留原始异常链，方便服务端排查真正原因，同时上层只需要理解统一的业务异常。

17.3 AgentOutputError

该异常表示模型已经返回响应，但响应不能作为合法的结构化数据使用，例如：

- 返回普通说明文字，没有 JSON；
- 顶层数据是数组，但程序要求对象；
- JSON 缺少 `city`、`days` 或其他必要字段；
- 字段类型不符合 `TripPlan`；
- JSON 可以解析，但无法通过 Pydantic 校验。

它和 `ExternalServiceError` 的区别是：

ExternalServiceError: 没有正常获得模型结果
AgentOutputError: 获得了结果, 但结果结构不合格

最终规划响应会依次经过:

```
Planner Agent 字符串
  ↓ extract_json_value()
Python dict
  ↓ TripPlan.model_validate()
TripPlan
```

任何 JSON 提取或 Pydantic 结构校验错误都会转换成 `AgentOutputError`, 不再回退到虚构行程。

17.4 PlanValidationError

该异常表示数据格式可能完全合法, 但内容违反了业务规则。

当前景点选择阶段已经覆盖:

- Agent 没有选择任何景点;
- Agent 返回了重复 POI ID;
- Agent 返回了候选集合之外的 POI ID。

例如下面的数据虽然是合法 JSON, 但不能被系统接受:

```
{
  "attractions": [
    {"poi_id": "UNKNOWN_ID", "reason": "推荐", "suggested_duration": 120}
  ]
}
```

因为 `UNKNOWN_ID` 不属于高德提供的真实候选集合, 所以程序会抛出 `PlanValidationError`。

这说明:

结构合法不等于业务可信, Pydantic 校验之后仍然需要业务规则校验。

18. 异常如何在不同层之间传播?

异常处理被分成了两个主要层次。

18.1 Agent 编排层负责识别错误性质

相关文件:

```
backend/app/agents/trip_planner_agent.py
```

该层了解 MCP、Agent、POI 和计划校验，因此由它判断错误属于：

- 外部服务失败；
- Agent 输出错误；
- 计划约束错误。

它不负责决定 HTTP 状态码，因为 Agent 编排本身不应该依赖 Web 协议。

18.2 FastAPI 路由层负责转换成 HTTP 响应

相关文件：

```
backend/app/api/routes/trip.py
```

路由层不重新分析 MCP 或 LLM 细节，只把业务异常映射成适合前端处理的 HTTP 响应：

业务异常	HTTP 状态	错误代码	用户含义
ExternalServiceError	503	EXTERNAL_SERVICE_UNAVAILABLE	外部服务暂时不可用
AgentOutputError	502	AGENT_OUTPUT_INVALID	模型结果未通过结构校验
PlanValidationError	502	PLAN_VALIDATION_FAILED	行程未通过可靠性校验
TripPlanningError	500	TRIP_PLANNING_FAILED	未进一步分类的规划失败
其他 Exception	500	INTERNAL_ERROR	未预料的系统内部错误

完整传播路径如下：



18.3 为什么最后仍要保留 except Exception？

即使已经定义了业务异常，仍可能出现未预料的程序错误，例如属性名写错、空对象访问或第三方库行为变化。

最后的 `except Exception` 是安全兜底，但它必须放在所有具体异常之后：

```
except ExternalServiceError:
    ...
except AgentOutputError:
    ...
except PlanValidationError:
    ...
except TripPlanningError:
    ...
except Exception:
    ...
```

如果把 `except Exception` 放在最前面，具体异常会先被它捕获，后面的分类分支永远没有机会执行。

19. 为什么不把原始异常直接返回前端？

旧代码会使用：

```
detail=f"生成旅行计划失败：{str(error)}"
```

这种写法方便开发调试，但不适合作为稳定接口，因为原始异常可能包含：

- 本地文件路径；
- MCP 启动命令；
- 第三方接口返回内容；
- 模型原始响应；
- 内部类名和实现细节；
- 将来可能出现的敏感数据。

现在 API 返回稳定结构：

```
{
  "detail": {
    "code": "AGENT_OUTPUT_INVALID",
    "message": "模型生成结果未通过结构校验，请重新生成"
  }
}
```

其中：

- `code` 方便前端和日志系统识别错误类型；
- `message` 用于直接展示给用户；
- 原始异常只保留在服务端异常链中。

20. 前端如何兼容结构化错误？

相关文件：

```
frontend/src/services/api.ts
```

后端的 `detail` 既可能是旧式字符串，也可能是新的对象。因此前端进行了兼容判断：

```
detail 是字符串 → 直接显示
detail 是对象   → 显示 detail.message
没有 detail     → 使用请求错误或默认提示
```

这样既支持新的结构化异常，也不会因为其他接口仍返回字符串而出现 `[object Object]`。

当旅行规划失败时，首页会停留在当前页面并显示错误，不会把失败响应保存到 `sessionStorage`，也不会跳转到结果页面。

21. 天气异常为什么可以降级，景点异常却不能？

不同数据在业务中的重要程度不同，因此不能机械地让所有异常都终止流程。

21.1 天气是可选增强信息

天气查询失败后，系统仍然可以使用经过验证的景点生成基础行程。因此当前策略是：

```
天气失败
  ↓
记录警告
  ↓
weather_info 使用空列表
  ↓
继续生成行程
  ↓
最终覆盖模型可能补造的天气
```

这种行为叫作“降级”：部分能力不可用，但剩余结果仍然可以保持可信。

21.2 景点是行程的核心事实

如果真实 POI 搜索失败，系统就没有可信景点名称、地址和坐标。此时继续生成只会迫使 LLM 编造事实，因此必须终止流程并抛出 `ExternalServiceError`。

判断是否可以降级，应当回答两个问题：

1. 去掉这部分数据后，剩余结果是否仍然有实际价值？
2. 继续运行是否会诱导模型创造本应来自外部数据源的事实？

如果第二个问题的答案是“会”，就不应降级为模型生成。

22. 新增测试验证了什么？

相关文件：

```
backend/tests/test_trip_failure_semantics.py
backend/tests/test_mcp_response_parser.py
```

当前共运行 9 个测试并全部通过。

原有 7 个测试验证 MCP JSON 解析器能够：

- 接受现有 `dict`；
- 接受现有 `list`；
- 从带前缀文本提取 JSON 对象；
- 从带前缀文本提取 JSON 数组；
- 拒绝空字符串；
- 拒绝没有 JSON 的错误文本；
- 拒绝不支持的数据类型。

新增 2 个测试验证最终规划失败语义：

22.1 拒绝非 JSON 的 Planner 响应

输入：

```
这不是合法JSON
```

预期：

```
抛出 AgentOutputError
不返回 TripPlan
不生成备用行程
```

22.2 拒绝结构不完整的 TripPlan

输入虽然是合法 JSON，但缺少必要字段：

```
{
  "city": "北京",
  "days": []
}
```

预期：

Pydantic 校验失败
转换为 AgentOutputError
不返回不完整计划

这两个测试证明了最关键的安全边界：模型输出不合格时，系统会明确失败，而不是悄悄替换成虚构数据。

23. 当前异常处理仍有哪些待完善项？

本阶段完成了异常体系的基础框架，但仍有一些路径需要后续统一。

23.1 最终行程的部分约束仍使用 ValueError

每日景点数量、景点缺少 POI ID、最终计划使用未知 POI、景点重复安排等检查，目前仍有部分使用普通 ValueError。

后续建立独立的行程校验器时，应统一转换成 PlanValidationError，使 API 返回 PLAN_VALIDATION_FAILED，而不是通用内部错误。

23.2 酒店 Agent 调用还需要外部服务异常包装

景点筛选模型和最终规划模型已经包装为 ExternalServiceError，酒店 Agent 调用仍需要采用同样策略。

23.3 天气降级还没有结构化警告字段

当前天气失败只记录日志并返回空天气。后续可以在响应中增加：

```
{
  "warnings": [
    {
      "code": "WEATHER_UNAVAILABLE",
      "message": "天气服务暂时不可用，本次行程未包含天气信息"
    }
  ]
}
```

这样用户可以区分“旅行日期超出预报范围”和“天气服务本次调用失败”。

23.4 还缺少 API 异常映射测试

当前新增测试覆盖了 _parse_response() 的失败语义，但尚未自动验证：

- ExternalServiceError 是否一定映射为 HTTP 503；
- AgentOutputError 是否一定映射为 HTTP 502；
- PlanValidationError 是否一定映射为 HTTP 502；
- 前端是否正确显示 detail.message。

这些内容适合在后续接口测试和前端测试中补充。

24. 异常改造带来的核心收益

24.1 失败结果不再污染业务数据

系统不会为了保持页面完整而生成虚构景点和坐标。

24.2 前端能够区分失败类型

稳定错误代码为重试、提示、监控和统计提供了基础。

24.3 调试范围更明确

看到 `AGENT_OUTPUT_INVALID` 时，应检查 Prompt、模型输出和 Pydantic 模型；看到 `EXTERNAL_SERVICE_UNAVAILABLE` 时，应检查 MCP、网络和第三方服务。

24.4 为后续可观测性奠定基础

未来可以按错误代码统计：

- 哪个外部服务最容易失败；
- 哪个 Agent 最常返回非法结构；
- 哪条业务约束最容易被违反；
- 每种失败是否值得自动重试。

24.5 面试中可以说明可靠性设计

本次修改体现的重点不只是“增加了几个异常类”，而是建立了完整边界：

外部事实由工具提供
结构由 Pydantic 检查
业务规则由程序验证
异常由业务层分类
HTTP 由接口层表达
前端只展示可信成功结果

这套思路同样适用于金融数据、库存、票价、医疗信息等不能由 LLM 自行补造的场景。